



SERRMA

Aplicacion multiplataforma para la gestion de recogida de
aceite

Documento de apoyo para la exposicion del TFG

Tecnologias	Flutter, Dart, Firebase y Cloud Firestore
Roles	Cliente, operario y encargado
Objetivo	Digitalizar solicitudes, rutas y estados de recogida
Enfoque	Beta funcional conectada a una base de datos en la nube

1. Introduccion

Este documento resume los puntos principales de la aplicacion desarrollada para el TFG: una solucion multiplataforma para gestionar solicitudes de recogida de residuos de aceite entre clientes, operarios y encargado.

SERRMA digitaliza un proceso que puede depender de llamadas, mensajes o registros manuales. Con la app, el cliente solicita una recogida y el operario consulta las solicitudes asignadas a su ruta, las marca como recogidas y mantiene una trazabilidad basica del servicio.

2. Problema que resuelve

En una empresa de recogida de residuos, la coordinacion entre clientes y trabajadores puede generar retrasos, duplicidades o falta de informacion actualizada. El sistema reduce estos problemas mediante:

- Registro centralizado de clientes.
- Solicitud digital de recogidas.
- Asignacion automatica de solicitudes por ruta.
- Consulta en tiempo real para operarios.
- Diferenciacion visual entre recogidas pendientes y realizadas.
- Acceso especial de encargado para consultar todas las rutas y gestionar clientes.

3. Objetivos del proyecto

- Crear una aplicacion funcional para una situacion real de gestion de residuos.
- Separar claramente los perfiles de cliente, operario y encargado.
- Utilizar una arquitectura sencilla, mantenible y adecuada para una version beta.
- Aprovechar tecnologias modernas que permitan evolucionar el proyecto.
- Dejar una base preparada para autentificacion real, permisos, notificaciones y estadisticas.

4. Tecnologias utilizadas

Flutter

Flutter se ha elegido porque permite desarrollar para varias plataformas desde una unica base de codigo. Esto encaja con una empresa que puede necesitar usar la app en movil, tablet o web sin mantener aplicaciones separadas.

- Desarrollo multiplataforma con un unico codigo Dart.
- Interfaz consistente en distintos dispositivos.
- Alta productividad gracias al sistema de widgets.
- Integracion sencilla con Firebase.
- Buena base para evolucionar la beta hacia Android, iOS o web.

Firestore y Cloud

Firebase actua como backend gestionado. En lugar de crear y mantener un servidor propio, la aplicacion guarda clientes, operarios y solicitudes en Cloud Firestore.

- Base de datos en la nube sin administrar infraestructura.
- Sincronizacion en tiempo real mediante streams.
- Integracion directa con Flutter.
- Escalabilidad adecuada para una app de gestion.
- Posibilidad de anadir Authentication, reglas de seguridad, Cloud Functions y notificaciones.

5. Arquitectura general

Archivo o carpeta	Responsabilidad
main.dart	Inicializa Flutter, Firebase y la carga de datos iniciales.
app.dart	Configura MaterialApp, tema y rutas principales.
presentation/screens	Contiene las pantallas de login, cliente y operario.
presentation/routes	Centraliza las rutas de navegacion.
models	Define las entidades principales del dominio.
services/firestore_database.dart	Agrupar las operaciones contra Cloud Firestore.

6. Modelo de datos

- **Clientes:** CIF, nombre, email, telefono, direccion, ruta, contacto, tipo de negocio, horario, contenedor, observaciones y estado activo.
- **Solicitudes de recogida:** cliente, ruta, bidones, capacidad, filtros, jabon, estado, operario asignado y fechas de creacion y recogida.
- **Operarios y rutas:** cinco operarios asignados a norte, sur, oeste, este y centro; el encargado puede ver todas las rutas.

7. Funcionalidades implementadas

Rol	Funcionalidades
Cliente	Inicia sesion, solicita recogidas, indica bidones, capacidad, filtros y jabon.
Operario	Consulta su ruta, filtra solicitudes, ve datos completos y marca recogidas como realizadas.
Encargado	Consulta todas las rutas, crea clientes, elimina solicitudes de prueba y supervisa contadores.

8. Flujo principal de uso

1. El cliente inicia sesion.
2. Solicita una recogida e introduce los datos necesarios.
3. La solicitud se guarda en Firestore con estado pendiente.
4. El operario asignado a la ruta consulta la solicitud en la app.
5. La solicitud aparece destacada visualmente como pendiente.
6. El operario realiza la recogida y la marca como completada.
7. El estado cambia a Recogido y se registra la fecha de recogida.
8. El encargado puede consultar el historico y el estado global.

9. Justificacion tecnica

La combinacion de Flutter y Firebase es adecuada porque resuelve dos necesidades clave: crear una interfaz multiplataforma y disponer de una base de datos en tiempo real sin desarrollar un backend completo desde cero.

Flutter aporta rapidez de desarrollo, una experiencia visual uniforme y facilidad para crear formularios, listas, dialogos y navegacion. Firebase aporta persistencia, sincronizacion y una base preparada para crecer.

Otra decision importante ha sido separar el acceso a datos en **FirestoreDatabase**. Esto evita que todas las pantallas conozcan directamente los detalles de Firestore y mejora la mantenibilidad.

10. Aspectos destacables para la exposicion

- La app esta conectada a una base de datos real en la nube.
- Las solicitudes se actualizan en tiempo real mediante streams.
- Existe asignacion automatica de rutas a operarios.
- Se diferencia el rol de cliente, operario y encargado.
- La interfaz muestra estados visuales: pendiente en rojo y recogido en verde.
- Se registra trazabilidad basica: fecha de solicitud y fecha de recogida.
- La estructura del codigo esta separada en modelos, servicios y pantallas.
- El proyecto se presenta como beta funcional con margen claro de evolucion.

11. Limitaciones de la version beta

Reconocer estas limitaciones durante la exposicion es positivo, porque demuestra criterio tecnico y capacidad de analizar la evolucion del proyecto.

- No usa Firebase Authentication real por usuario.
- Las contraseñas son simples y estan pensadas para la demostracion.
- Las reglas de Firestore son temporales y deben endurecerse.
- No existe gestion avanzada de permisos por documento.
- No hay notificaciones push.
- No incluye panel de estadisticas ni exportacion de informes.

12. Mejoras futuras

- Incorporar Firebase Authentication.
- Definir reglas de seguridad por rol y usuario.
- Crear notificaciones push para avisar a operarios.

- Anadir modo offline para zonas con mala cobertura.
- Implementar estadisticas por ruta, fecha y litros recogidos.
- Exportar historicos en PDF o Excel.
- Crear un panel web de administracion para el encargado.

13. Conclusion

SERRMA demuestra como una aplicacion multiplataforma puede mejorar un proceso real de gestion de residuos. El proyecto digitaliza la comunicacion entre cliente, operario y encargado, centraliza la informacion en la nube y ofrece una base funcional sobre la que seguir creciendo.

La eleccion de Flutter y Firebase permite presentar una solucion moderna, escalable y realista para una empresa que necesita movilidad, rapidez de desarrollo y datos actualizados en tiempo real.

Documento generado como apoyo a la defensa del TFG.